

测试版水印与软件防破解方案

文档版本：v1.0

编写日期：2026 年 3 月 14 日

开发负责人：王舟

一、需求分析

1.1 水印需求

类型	用途	优先级
屏幕水印	防止测试版截屏外传	P0
导出水印	防止导出文件外流	P0
图片水印	防止图片盗用	P1

1.2 防破解需求

功能	用途	优先级
许可证验证	证明软件使用权	P0
签名验证	防止许可证伪造	P0
机器绑定	防止一证多用	P0
有效期检查	控制使用时间	P0
在线验证	定期联网查票	P1

二、水印实现方案

2.1 屏幕水印

实现原理

- 创建透明覆盖层
- 平铺半透明文字
- 倾斜 45 度防 PS
- 鼠标穿透不影响操作

代码实现

```
// frontend/src/components/watermark.rs
use dioxus::prelude::*;

#[component]
pub fn WatermarkOverlay(
    text: String,
    is_visible: bool,
) -> Element {
    if !is_visible {
        return rsx! {};
    }

    rsx! {
        div {
            style: "
                position: fixed;
                top: 0;
                left: 0;
                width: 100vw;
                height: 100vh;
                pointer-events: none;
                z-index: 99999;
                overflow: hidden;
            ",

            for i in 0..15 {
                for j in 0..15 {
                    div {
                        style: format!(
                            "
                                position: absolute;
                                left: {}px;
                                top: {}px;
                                transform: rotate(-45deg);
                                font-size: 20px;
                                color: rgba(100, 100, 100, 0.15);
                                font-weight: bold;
                                user-select: none;
                                white-space: nowrap;
                            "
                        );
                    }
                }
            }
        }
    }
}
```



```

        100 400 Td
        0.2 0.2 0.2 scn
        ({} ) Tj
        ET
        Q
        "#,
        watermark_text
    );

    // 添加到页面
    let page = doc.get_object_mut(*page_ref)?;
    // ... 添加水印到页面内容
}

doc.save(output_pdf)?;
Ok(())
}

```

2.3 Excel 导出水印

```

// src/services/excel_watermark.rs
use rust_xlsxwriter::*;

pub fn add_watermark_to_excel(
    output_path: &str,
    watermark_text: &str,
) -> Result<()> {
    let mut workbook = Workbook::new();
    let worksheet = workbook.add_worksheet();

    // 添加水印文本
    let format = Format::new()
        .set_font_color(Color::RGB(0xD3D3D3))
        .set_font_size(12);

    worksheet.write_string_with_format(0, 0, watermark_text, &format)?;

    workbook.save(output_path)?;
}

```

```
Ok(()))  
}
```

三、许可证系统

3.1 许可证数据结构

```
{  
  "license_type": "trial",  
  "user_email": "user@example.com",  
  "user_name": "张三",  
  "issued_at": "2026-03-14T10:00:00Z",  
  "expires_at": "2026-04-14T10:00:00Z",  
  "machine_id": "abc123xyz",  
  "signature": "MEUCIQDxxx...yyy"  
}
```

3.2 许可证类型

类型	有效期	功能	水印
测试版	30 天	基础功能	有
标准版	1 年	全部功能	无
企业版	永久	全部功能 + 定制	无

3.3 许可证生成（服务器端）

```
use ring::{signature, rand};  
use base64::{Engine as _, engine::general_purpose};  
  
const PRIVATE_KEY_PKCS8: &[u8] = include_bytes!("private_key.pkcs8");  
  
pub fn generate_license(  
    license_type: &str,  
    user_email: &str,  
    days: i64,  
    machine_id: &str,  
) -> Result<LicenseInfo> {  
    let now = chrono::Utc::now();
```

```

// 准备数据
let data = format!(
    "{}: {}: {}: {}: {}",
    license_type,
    user_email,
    now.timestamp(),
    (now + chrono::Duration::days(days)).timestamp(),
    machine_id,
);

// 私钥签名
let key_pair = signature::Ed25519KeyPair::from_pkcs8(PRIVATE_KEY_PKCS8);
let sig = key_pair.sign(data.as_bytes());

Ok(LicenseInfo {
    license_type: license_type.to_string(),
    user_email: user_email.to_string(),
    issued_at: now,
    expires_at: Some(now + chrono::Duration::days(days)),
    machine_id: machine_id.to_string(),
    signature: general_purpose::STANDARD.encode(sig.as_ref()),
})
}

```

3.4 许可证验证（客户端）

```

use ring::signature;
use base64::{Engine as _, engine::general_purpose};

const PUBLIC_KEY_PEM: &str = r#"
-----BEGIN PUBLIC KEY-----
MCowBQYDK2VwAyEA... (公钥)
-----END PUBLIC KEY-----
"#;

pub fn verify_license(license: &LicenseInfo) -> bool {
    // 1. 重建数据
    let data = format!(
        "{}: {}: {}: {}: {}",

```

```

        license.license_type,
        license.user_email,
        license.issued_at.timestamp(),
        license.expires_at.map(|e| e.timestamp()).unwrap_or(0),
        license.machine_id,
    );

// 2. 解码签名
let sig = match general_purpose::STANDARD.decode(&license.signature) {
    Ok(s) => s,
    Err(_) => return false,
};

// 3. 公钥验证
let public_key = parse_pem_public_key(PUBLIC_KEY_PEM).unwrap();
let unparsed_pk = signature::UnparsedPublicKey::new(
    &signature::ED25519,
    public_key,
);

unparsed_pk.verify(data.as_bytes(), &sig).is_ok()
}

```

四、机器码绑定

4.1 获取机器唯一标识

```

// src/utils/machine_id.rs
use machine_uid::get_machine_uid;
use sha2::{Sha256, Digest};

pub fn get_machine_id() -> Result<String> {
    let raw_id = get_machine_uid()?;

    let mut hasher = Sha256::new();
    hasher.update(raw_id.as_bytes());
    let hash = hasher.finalize();
}

```

```
    Ok(format!("{:x}", hash))
}
```

4.2 绑定验证

```
pub fn verify_machine_binding(license: &LicenseInfo) -> bool {
    let current_machine_id = get_machine_id().unwrap_or_default();
    license.machine_id == current_machine_id
}
```

五、许可证管理器

5.1 本地存储

```
~/.config/legalminds/
└─ license.json
```

5.2 管理器实现

```
// src/license/manager.rs
use std::path::PathBuf;
```

```
pub struct LicenseManager {
    license: Option<LicenseInfo>,
    license_path: PathBuf,
}
```

```
impl LicenseManager {
    pub fn new() -> Result<Self> {
        let license_path = dirs::config_dir()
            .map(|d| d.join("legalminds").join("license.json"))
            .ok_or_else(|| anyhow!("无法找到配置目录"))?;

        Ok(Self { license: None, license_path })
    }
}
```

```
/// 加载许可证
```

```
pub fn load_license(&mut self) -> Result<()> {
    if self.license_path.exists() {
        let content = std::fs::read_to_string(&self.license_path)?;
```

```

        let license: LicenseInfo = serde_json::from_str(&content)?;

        if verify_license(&license) {
            self.license = Some(license);
            Ok(())
        } else {
            Err(anyhow!("许可证验证失败"))
        }
    } else {
        Err(anyhow!("未找到许可证文件"))
    }
}

/// 激活许可证
pub fn activate(&mut self, license_key: &str, email: &str) -> Result<LicenseInfo> {
    let machine_id = get_machine_id()?;

    // 调用服务器激活
    let license = activate_online(license_key, email, &machine_id).await;

    // 验证
    if !verify_license(&license) {
        return Err(anyhow!("许可证验证失败"));
    }

    // 保存
    self.save_license(&license)?;
    self.license = Some(license.clone());

    Ok(license)
}

/// 检查状态
pub fn check_status(&self) -> LicenseStatus {
    match &self.license {
        None => LicenseStatus::NotFound,
        Some(license) => {
            if !verify_license(license) {
                LicenseStatus::Invalid
            } else if license.is_expired() {

```

```

        LicenseStatus::Expired
    } else if !verify_machine_binding(license) {
        LicenseStatus::MachineMismatch
    } else {
        LicenseStatus::Valid(license.clone())
    }
    }
}
}

/// 获取水印文本
pub fn get_watermark_text(&self) -> String {
    match &self.license {
        Some(license) if license.is_trial() => {
            format!("🔒 测试版 - {} - 有效期至: {}",
                license.user_email,
                license.expires_at
                    .map(|e| e.format("%Y-%m-%d").to_string())
                    .unwrap_or_else(|| "未知".to_string())
            )
        }
        _ => String::new(),
    }
}
}

```

六、激活界面

6.1 前端组件

```

// frontend/src/license.rs
#[component]
pub fn LicenseModal(on_close: EventHandler<()>) -> Element {
    let mut license_key = use_signal(|| String::new());
    let mut email = use_signal(|| String::new());
    let mut error = use_signal(|| Option::<String>::None);

    let handle_activate = move |_| {
        let key = license_key();
    }
}

```

```

let mail = email();

spawn(async move {
    match activate_license(&key, &mail).await {
        Ok(_) => on_close.call(()),
        Err(e) => error.set(Some(e.to_string())),
    }
});

};

rsx! {
    div { class: "modal-overlay",
        div { class: "modal license-modal",
            h2 { "激活许可证" }

            if let Some(err) = error() {
                div { class: "error-message", {err} }
            }

            input {
                type: "email",
                placeholder: "注册邮箱",
                value: email(),
                oninput: move |e| email.set(e.value()),
            }

            input {
                type: "text",
                placeholder: "许可证密钥",
                value: license_key(),
                oninput: move |e| license_key.set(e.value()),
            }

            button {
                class: "primary",
                onclick: handle_activate,
                "激活"
            }

            div { class: "trial-info",

```

```
h4 { "📄 测试版说明" }
ul {
  li { "测试有效期: 30 天" }
  li { "功能限制: 部分高级功能不可用" }
  li { "导出文件带有测试版水印" }
  li { "仅限个人评估使用" }
}
}
```

七、安全等级评估

措施	安全等级	成本	推荐
屏幕水印	★★	低	✅ 必做
导出水印	★★	低	✅ 必做
许可证签名	★★★★	中	✅ 必做
机器绑定	★★★	中	✅ 推荐
在线验证	★★★★	中	⚠️ 二期做
代码混淆	★★	低	⚠️ 可选

八、实施计划

8.1 测试版（1 周）

任务	工期	状态
屏幕水印	1 天	🕒
导出水印	2 天	🕒
许可证验证	2 天	🕒
机器绑定	1 天	🕒
激活界面	1 天	🕒

8.2 正式版（再加 2 周）

任务	工期	状态
在线验证	3 天	
离线宽限期	2 天	
许可证服务器	5 天	
反调试	2 天	

九、依赖库

```
# Cargo.toml
[dependencies]
# 水印
lopdf = "0.34"           # PDF 处理
rust_xlsxwriter = "0.58" # Excel 处理
image = "0.25"           # 图片处理

# 许可证
ring = "0.17"            # 加密签名
base64 = "0.21"          # Base64 编码
machine_uid = "0.2"       # 机器码

# 其他
dirs = "5"               # 配置目录
serde_json = "1"         # JSON 处理
```

十、常见问题

Q1: 水印能去掉吗？

技术上能，但需要修改代码。有许可证验证的话，改了也打不开软件。

Q2: 许可证能复制吗？

不能。绑定了机器码，复制到其他电脑验证不通过。

Q3: 离线能用吗？

可以。测试版 30 天内无需联网。正式版可设置 7 天离线宽限期。

Q4: 被破解了怎么办?

1. 在线验证可远程吊销
2. 下次更新加强验证
3. 法律手段 (如有商业损失)

十一、总结

推荐方案

测试版 = 屏幕水印 + 导出水印 + 许可证验证 + 机器绑定
(约 1 周, 安全性★★★★★)

正式版 = 测试版 + 在线验证 + 离线宽限
(再加 2 周, 安全性★★★★★★)

核心思路

问题 解决方案 类比

防止滥用 水印 盖"测试版"章

防止复制 签名验证 电影票防伪

限制设备 机器绑定 实名制

控制时间 有效期 场次时间

文档版本 : v1.0

最后更新: 2026 年 3 月 14 日

开发负责人: 王舟

邮箱 : main@mails.wedevs.org

电话: 15378391447